

M1 - UE MU4IN401
ARCHITECTURE AVANCÉE DES NOYAUX DES
SYSTÈMES D'EXPLOITATION

Sept. 2024

Equipe pédagogique :

DUBOIS Swan (Swan.Dubois@lip6.fr)

SENS Pierre (Pierre.Sens@lip6.fr)

SOPENA Julien (Julien.Sopena@lip6.fr)

Table des matières

TD 1 : Rappels systèmes et introduction au Noyau Unix	2
TD 2 : La synchronisation des processus	6
Fichier slp.c	8
TD 3 : Les signaux	10
Fichier sig.c	12
TD 4 : La gestion du temps et l'ordonnancement des processus	15
Fichier clock2b.c	17
TD 5 : Commutation de processus	21
Fichier swtch.c	23
TD 6 : Création et terminaison de processus	25
Fichier fork.c	27
Fichier exit.c	30
TD 7 : Le swap	31
Fichier sched.c	32
TD 8 : Le buffer cache	36
Fichier bio2.c	37
TD 9 : Representation Interne des Fichiers	44
Fichier iget.c	46
Fichier namei.c	49
TD 10 : Structure des fichiers	
Traduction d'adresse / Gestion de l'espace libre sur disque	53
Fichier subr.c	55
Fichier alloc.c	58
TD 11 : Complément sur les Entrées-Sorties	61
Fichier revision.c	64
Annexes :	65
Fichier buf.h	66
Fichier callo.h	68
Fichier conf.h	69
Fichier fblk.h	70
Fichier filsys.h	71
Fichier inode.h	72
Fichier inode.hv6	73
Fichier ino.hv6	74
Fichier file.h	75
Fichier mount.h	76
Fichier param.h	77
Fichier proc.h	78
Fichier signal.h	80
Fichier text.h	81
Fichier tty.h	82
Fichier types.h	83
Fichier user.h	84
Fichier var.h	86

TD 1 - RAPPELS SYSTÈMES ET INTRODUCTION AU NOYAU UNIX

Vous allez étudier le code d'un Unix version 6 à 7 pendant le module Noyau. Vous pouvez trouver les sources qui n'ont pas été reportées dans ce fascicule à l'URL suivante : <http://v6.cuzuco.com/v6.pdf>.

Prérequis

Vous devez être familiarisé avec la programmation en C.

Rappel programmation C

Le code étudié en TD est un unix V6 de 1977 écrit par K. Thompson et D. Ritchie dans la première version du langage C (K&R C) non ANSI.

Voici les principales différences syntaxiques entre le K&R C et le C ANSI :

	K&R C	C ANSI	Commentaires
Fonctions	<pre>f(a,b,c) int *a; char b; { return(b); }</pre>	<pre>int f(int *a, char b, int c) { return(b); }</pre>	Par défaut en K&R C les fonctions retournent des int, des paramètres non déclarés sont implicitement des int (par exemple la variable c)
Opérateurs d'affectation composée	<code>a -= 10;</code>	<code>a -= 10;</code>	En C ANSI <code>-=</code> , <code>+=</code> , <code>= </code> ...sont remplacés par <code>-=</code> , <code>+=</code> , <code> =</code>
Utilisation de la clause register	<pre>register a; register char *b;</pre>	<pre>register int a; register char *b;</pre>	Très utilisé dans le code du Noyau pour indiquer le stockage d'une variable si possible dans un registre.

Question 1

Quel est l'intérêt de déclarer une variable en "register" ?

Question 2

Que fait le programme suivant :

```
f(from, to, count)
int *from, *to;
register count;
{
    register *f, *t;

    f = from;
    t = to;
    do
        *t++ = *f++;
    while(--count);
}
```

Question 3

Soit un tableau t d'une structure quelconque x :

```
struct x {
int x_a;
} t[MAX];
```

Le code suivant parcourt t.

```
int i,cpt;
cpt = 0;
for (i=0; i < MAX; i++)
    cpt += t[i].x_a;
```

Réécrivez ce code en utilisant à la place de la variable i, une variable `struct x *p`.

Rappels sur le matériel

Question 4

Qu'est ce que le CPU et quel est son rôle? Que sont les registres? Qu'est ce que la mémoire vive (ou RAM)? La mémoire morte? Qu'est ce qu'une interruption? Comment le CPU communique avec le matériel? Qu'est ce que le DMA et quel est son utilité?

Question 5

Qu'est ce que la mémoire virtuelle?

Question 6

Qu'est ce que commuter? Comment peut-on commuter?

Question 7

Que sont les segments de données, de code et de pile? Comment le CPU y accède? Décrivez dans le petit programme suivant l'état de la mémoire au point indiqué.

```
int x = 37;

void f() {
    int i = 17;

    printf("Hello: %d\n", i);

    i = 45;

    // point d'exécution
}

void main() {
    f();
}
```

Question 8

Quelles sont les différences entre exécution en mode *usager* et en mode *système*? Pourquoi y-a-t-il ces différences?

Question 9

Rappelez ce qu'est ce qu'un appel système.

Question 10

Rappelez ce qu'est une interruption matériel.

Question 11

Justifiez la présence d'une pile d'exécution pour l'utilisateur et d'une pile pour le système.

Question 12

Etudiez la structure *user* et la structure *proc*. Expliquez pourquoi le descripteur de processus est décomposé en deux structures.

Question 13

Qu'est ce qu'est la zone U?

Question 14

Expliquez la présence d'un numéro d'identité (*pid*) pour chaque processus. Comment est-il attribué ?

Question 15

Ecrire une fonction `int nb_fils()` interne au noyau qui retourne le nombre de processus fils du processus courant.

TD 2 - LA SYNCHRONISATION DES PROCESSUS

But

Le but de ce TD est de comprendre l'implémentation des fonctions de synchronisation `sleep` et `wakeup` et des fonctions de masquage des interruptions `sp1` et `gp1`.

Prérequis

Vous devez connaître la différence entre mode utilisateur et mode système et être familiarisés avec la table des processus.

Question 1

Rappelez le rôle des fonctions `sleep` et `wakeup`.

Question 2

Rappelez les opérations bit-à-bit en C. Que fait la fonction `fsig`?

Question 3

En utilisant les fonctions `sleep` et `wakeup` vues en cours, essayez de programmer une fonction `flock` qui verrouille un fichier en lecture/écriture (donc permet un accès exclusif au fichier). Si le fichier est déjà verrouillé, la fonction devra mettre le processus demandeur en attente. Écrire une fonction `frelease` qui libère le verrou, et réveille les éventuels processus qui attendraient de verrouiller ce fichier.

On utilisera le champ `i_flag` de l'inode correspondant au fichier, en particulier les valeurs `ILOCK` et `IWANT`. On s'endormira sur l'adresse de l'inode, avec le paramètre `PINOD`.

Dans `frelease`, on réveillera tous les processus endormis sur l'inode (car `i_flag` a le bit `IWANT` positionné à 1).

Que se passe-t-il si un processus ouvre le fichier et y accède sans avoir préalablement appelé `flock`? Pensez-vous que ce soit un réel problème?

Question 4

Quelles sont les opérations effectuées par les fonctions `sp1` et `gp1`? Quel en est l'effet?

En quelles circonstances faut-il utiliser en plus `sp1` ? Donnez des exemples.

Question 5

Lorsque vous appelez `sleep`, quand s'effectue la commutation de processus ? Même question lorsque vous appelez `wakeup`.

Dans quelles autres circonstances intervient la commutation de processus ?

Quel est le rôle de la variable `runrun` ?

Question 6

Qu'est-ce que le swapper ? Quel est le rôle des variables `runin` et `runout`.

Question 7

Que signifie l'argument `pri` de `sleep` ? Quand joue-t-il ?

Question 8

Décrivez l'algorithme des fonctions `sleep` et `wakeup`.

FICHER SLP.C

```

1  #include " ../ sys / param . h "
2  #include " ../ sys / types . h "
3  #include " ../ sys / user . h "
4  #include " ../ sys / proc . h "
5
6
7  char      runin , runout , runrun ;
8
9
10 /*
11  * Give up the processor till a wakeup occurs
12  * on chan, at which time the process
13  * enters the scheduling queue at priority pri.
14  * The most important effect of pri is that when
15  * pri<=PZERO a signal cannot disturb the sleep;
16  * if pri>PZERO signals will be processed.
17  * Callers of this routine must be prepared for
18  * premature return, and check that the reason for
19  * sleeping has gone away.
20  */
21
22
23 sleep(chan, pri)
24 caddr_t chan;
25 {
26     register struct proc *rp = u.u_procp;
27     register s, op;
28
29
30     if (pri > PZERO) {
31         if(issig())
32             goto psig;
33         spl(CLINHB);
34         rp->p_wchan = chan;
35         rp->p_stat = SSLEEP;
36         rp->p_pri = pri;
37         spl(NORMAL);
38         if(runin != 0) {
39             runin = 0;
40             wakeup(&runin);
41         }
42         swtch();
43         if(issig())
44             goto psig;
45     } else {
46         spl(CLINHB);
47         rp->p_wchan = chan;
48         rp->p_stat = SSLEEP;
49         rp->p_pri = pri;
50         spl(NORMAL);
51         swtch();
52     }
53     return;
54

```

```

55
56      /*
57      * If priority was low (>PZERO) and
58      * there has been a signal,
59      * execute non-local goto to
60      * the qsav location.
61      */
62      psig:
63          aretu(u.u_qsav);
64  }
65
66
67  /*
68  * Wake up all processes sleeping on chan.
69  */
70  wakeup(chan)
71  register caddr_t chan;
72  {
73      register struct proc *p;
74      register c, i ;
75
76
77      c = chan;
78      p = &proc[0];
79      i = NPROC;
80      do {
81          if(p->p_wchan == c) {
82              setrun(p);
83          }
84          p++;
85      } while(--i);
86  }
87
88
89  /*
90  * Set the process running;
91  * arrange for it to be swapped in if necessary.
92  */
93  setrun(p)
94  register struct proc *p;
95  {
96      p->p_wchan = 0;
97      p->p_stat = SRUN;
98      if(p->p_pri < u.u_procp->p_pri)
99          runrun++;
100      if(runout != 0 && (p->p_flag&SLOAD) == 0) {
101          runout = 0;
102          wakeup(&runout);
103      }
104  }

```

TD 3 - LES SIGNAUX

But

Le but de ce TD est de comprendre l'implémentation des signaux. L'implémentation étudiée ne prend pas en compte la gestion des signaux suivant la norme POSIX, en particulier, les signaux ne peuvent pas être masqués. De plus, la version étudiée ne propose pas le signal SIGCHLD.

On vous rappelle qu'un signal est un message envoyé par un processus ou par le noyau à un processus. Un processus enregistre des comportements associés au signal : le comportement ignorer le signal (SIG_IGN), le comportement exécuter le comportement par défaut (SIG_DFL) qui est bien souvent d'arrêter le processus ou un comportement personnalisé. Dans ce cas, il s'agit d'une fonction du processus. Un comportement personnalisé est souvent appelé gestionnaire ou *handler* en anglais. Pour associer un gestionnaire à un signal, on peut utiliser la fonction C `signal(int no, void (*handler)(int))`. Elle associe la fonction `handler` au signal `no`.

Prérequis

Vous devez avoir utilisé les primitives système `kill` et `signal`.

Vous devez en outre avoir compris les mécanismes de synchronisation à base de `sleep` et `wakeup`.

Question 1

Ecrivez un programme qui affiche la chaîne de caractère "Bonjour" toutes les secondes. Pour réaliser ce programme, vous utiliserez la fonction `int alarm(int nb_sec)` qui envoie un signal SIGALRM au processus au bout de `nb_sec` secondes.

Question 2

Quelle est la différence entre un signal ignoré et un signal masqué ?

Question 3

Quels sont les rôles des variables `p->p_sig` et `u.u_signal` (vous pouvez vous aider des codes de `ssig()` et `issig()`) ?

Pour quelle raison `u.u_signal` est dans la zone swappable ?

Question 4

Quelles sont les différences et similitudes entre les signaux sous Unix et les interruptions matérielles ?

Question 5

Quels sont les rôles des fonctions `kill`, `psignal`, `issig`, `psig`, `fsig`, `sendsig` et `ssig` ?

Question 6

Donnez les algorithmes des appels système `ssig` et `kill`.

Question 7

Expliquez comment se déroule l'émission d'un signal avec `psignal`. Expliquez ce qui se passe lorsque le processus récepteur est en attente d'une ressource système ?

Question 8

Expliquez quand et comment se déroule la réception d'un signal.

Que se passe-t-il en cas de réception de plusieurs signaux ?

Comment est réalisé l'appel de la fonction (handler) spécifiée par l'utilisateur ?

Question 9

Expliquez le code de `sendsig()`.

FICHER SIG.C

```

1
2  /*
3   * signal system call
4   */
5  ssig()
6  {
7      register a;
8
9      a = u.u_arg[0];
10     if(a<=0 || a>=NSIG || a==SIGKIL) {
11         u.u_error = EINVAL;
12         return;
13     }
14     u.u_ar0[R0] = u.u_signal[a];
15     u.u_signal[a] = u.u_arg[1];
16     u.u_procp->p_sig &= ~(1<<(a-1));
17 }
18
19 /*
20 * kill system call
21 */
22 kill()
23 {
24     register struct proc *p, *q;
25     register a;
26     int f;
27
28     f = 0;
29     a = u.u_arg[1];
30     q = u.u_procp;
31     for(p = &proc[0]; p < &proc[NPROC]; p++) {
32         if(p->p_stat == NULL)
33             continue;
34         if(a != 0 && p->p_pid != a)
35             continue;
36         if(a == 0 && (p->p_ttyp != q->p_ttyp || p <= &proc[1]))
37             continue;
38         if(u.u_uid != 0 && u.u_uid != p->p_uid)
39             continue;
40         f++;
41         psignal(p, u.u_arg[0]);
42     }
43     if(f == 0)
44         u.u_error = ESRCH;
45 }
46
47 /*
48 * Send the specified signal to
49 * the specified process.
50 */
51 psignal(p, sig)
52 register struct proc *p;
53 register sig;
54 {

```

```

55
56     if ((unsigned) sig >= NSIG)
57         return;
58     if (sig) {
59         p->p_sig |= 1<<(sig-1);
60         if (p->p_stat == SSLEEP && p->p_pri > PZERO)
61             setrun(p);
62     }
63 }
64
65 /*
66  * Returns true if the current
67  * process has a signal to process.
68  * This is asked at least once
69  * each time a process enters the
70  * system.
71  * A signal does not do anything
72  * directly to a process; it sets
73  * a flag that asks the process to
74  * do something to itself.
75  */
76 issig()
77 {
78     register n;
79     register struct proc *p;
80
81     p = u.u_procp;
82     while (p->p_sig) {
83         n = fsig(p);
84         if ((u.u_signal[n]&1) == 0)
85             return(n);
86         p->p_sig &= ~(1<<(n-1));
87     }
88     return(0);
89 }
90
91 /*
92  * Perform the action specified by
93  * the current signal.
94  * The usual sequence is:
95  * if(issig())
96  *   psig();
97  */
98 psig()
99 {
100     register n, p;
101     register struct proc *rp;
102
103     rp = u.u_procp;
104     n = fsig(rp);
105     if (n==0)
106         return;
107     rp->p_sig &= ~(1<<(n-1));
108     if ((p=u.u_signal[n]) != 0) {
109         u.u_error = 0;
110         u.u_signal[n] = 0;
111         sendsig(p, n);
112         return;
113     }

```

```
114     switch(n) {
115
116         case SIGQUIT:
117         case SIGINS:
118         case SIGTRC:
119         case SIGIOT:
120         case SIGEMT:
121         case SIGFPT:
122         case SIGBUS:
123         case SIGSEG:
124         case SIGSYS:
125             if(core())
126                 n += 0200;
127     }
128     exit(n);
129 }
130
131 /*
132  * find the signal in bit-position
133  * representation in p_sig.
134  */
135 fsig(p)
136 struct proc *p;
137 {
138     register n, i;
139
140     n = p->p_sig;
141     for(i=1; i<NSIG; i++) {
142         if(n & 1)
143             return(i);
144         n >>= 1;
145     }
146     return(0);
147 }
148
149 /* adapted from the version 6 */
150 sendsig(void *handler, int num) {
151     sp = u.u_ar0[SP] - 4;
152     grow(sp);
153     u.u_ar0[SP] = sp;
154     suword(sp, u.u_ar0[PC]); /* sp[0] = PC */
155     u.u_ar0[PC] = handler;
156 }
```


TD 4 - LA GESTION DU TEMPS ET L'ORDONNANCEMENT DES PROCESSUS

Le but de ce TD est de comprendre l'implémentation des fonctions de gestion du temps dans un système comme Unix et d'étudier un exemple de routine de traitement d'interruptions.

Prérequis

Vous devez avoir manipulé les primitives de gestion de l'horloge et d'ordonnancement des processus.

Question 1

Récapitulez les différentes notions du temps présentes dans le système.

Question 2

Que sont les *timeouts*? A quoi servent-ils? Quelle est la structure de données utilisée pour implémenter ces *timeouts*?

Question 3

Décrivez l'algorithme de la routine `timeout`. Expliquez brièvement comment fonctionne `delay`.

Question 4

En vous inspirant de l'algorithme utilisé par `timeout`, programmez la fonction `untimeout(ident)`, qui enlève de la table de `callout` l'entrée correspondante au `timeout` dont l'identifiant est `ident`. La fonction renvoie -1, si elle n'a pas trouvé l'identifiant. Sinon elle renvoie 0.

Question 5

Expliquez la structure de la routine d'interruption horloge et l'enchaînement des diverses fonctions. Décrivez l'algorithme de la fonction `clock` et le fonction `realtime`.

Question 6

Programmez la fonction `restart`. Elle doit appeler les fonctions dont les timeouts sont arrivés à expiration. Après cela, elle doit décaler les entrées correspondantes au timeouts en cours vers le début du vecteur de `callout`.

Question 7

Expliquez l'algorithme de la fonction *setpri*. Quand est-ce que cette fonction est appelée ?

FICHER CLOCK2B.C

```

1  #include " ../ sys / param . h "
2  #include " ../ sys / conf . h "
3  #include " ../ sys / proc . h "
4  #include " ../ sys / user . h "
5  #include " ../ sys / var . h "
6
7  /*
8   * clock is called straight from
9   * real time clock interrupt.
10  * Functions:
11  *     implement callouts
12  *     maintain user/system times
13  *     profile user proc's and kernel
14  *     lightning bolt wakeup.
15  */
16
17
18  clock ()
19  {
20      extern int iaflags , idleflag ;
21      register struct callo * p1 ;
22      register int * pc ;
23
24      if ( callout [ 0 ] . c_func != 0 ) {
25          p1 = & callout [ 0 ] ;
26          while ( p1 -> c_time <= 0 && p1 -> c_func != 0 )
27              p1 ++ ;
28          p1 -> c_time -- ;
29      }
30      if ( ! idleflag )
31      {
32          if ( u . u_procp -> p_cpu < 80 )
33              u . u_procp -> p_cpu ++ ;
34      }
35
36
37      if ( user_mode ( ) ) {
38          u . u_utime ++ ;
39      } else {
40          if ( ! idleflag )
41              u . u_stime ++ ;
42      }
43
44
45      if ( ( callout [ 0 ] . c_func != 0 && callout [ 0 ] . c_time <= 0 ) )
46          iaflags |= CALOUT ;
47      if ( ++ lbolt >= HZ )
48          iaflags |= WAKEUP ;
49  }
50
51
52  realtime ()
53  {
54      register struct proc * pp ;

```

```

55
56
57     lbolt -= HZ;
58     time++;
59
60
61     /* force a switch every second */
62     runrun++;
63     wakeup(&lbolt);
64     for (pp = &proc[0]; pp < &proc[NPROC]; pp++)
65         if (pp->p_stat) {
66             if (pp->p_time <= 127)
67                 pp->p_time++;
68
69
70             if(pp->p_clktim)
71                 if(--pp->p_clktim == 0)
72                     psignal(pp, SIGALRM);
73             /*
74             * Update CPU Usage info:
75             */
76             pp->p_cpu >>= 1;
77             if (pp->p_pri >= PUSER)
78                 setpri(pp);
79
80
81         }
82     if (runin != 0) {
83         runin = 0;
84         wakeup((caddr_t)&runin);
85     }
86 }
87
88
89
90 /*
91 * timeout is called to arrange that
92 * fun(arg) is called in tim/HZ seconds.
93 * An entry is sorted into the callout
94 * structure. The time in each structure
95 * entry is the number of HZ's more
96 * than the previous entry.
97 * in this way, decrementing the
98 * first entry has the effect of
99 * updating all entries.
100 */
101 timeout(fun, arg, tim)
102 int (*fun)();
103 caddr_t arg;
104 int tim;
105 {
106     register struct callo *p1, *p2;
107     int ps;
108     register int t;
109
110     t = tim;
111     p1 = &callout[0];
112     ps = gpl();
113     spl(CLINHB);

```

```

114     while(p1->c_func != 0 && p1->c_time <= t) {
115         t -= p1->c_time;
116         p1++;
117     }
118     p1->c_time -= t;
119     p2 = p1;
120     while(p2->c_func != 0)
121         p2++;
122     if(p2 == &callout[NCALL - 1]) {
123         spl(ps);
124         panic("no_callout_space");
125     }
126     while(p2 >= p1) {
127         (p2+1)->c_time = p2->c_time;
128         (p2+1)->c_func = p2->c_func;
129         (p2+1)->c_arg = p2->c_arg;
130         p2--;
131     }
132     p1->c_time = t;
133     p1->c_func = fun;
134     p1->c_arg = arg;
135     spl(ps);
136 }
137
138
139 restart()
140 {
141     struct callo *p1;
142     struct callo *p2;
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160 }
161
162
163 #define PDELAY (PZERO-1)
164 delay(ticks)
165 {
166     extern wakeup();
167
168
169     if (ticks <= 0)
170         return;
171     timeout(wakeup, (caddr_t)u.u_procp+1, ticks);
172     sleep((caddr_t)u.u_procp+1, PDELAY);

```

```
173 }
174
175 /*
176  * Set user priority.
177  * The rescheduling flag (runrun)
178  * is set if the priority is higher
179  * than the currently running process.
180  */
181 setpri(up)
182 {
183     register *pp, p;
184
185     pp = up;
186     p = (pp->p_cpu & 0377)/16;
187     p += PUSER + pp->p_nice;
188     if(p > 127)
189         p = 127;
190     if(p < u.u_procp->p_pri)
191         runrun++;
192     pp->p_pri = p;
193 }
```

TD 5 - COMMUTATION DE PROCESSUS

But

Le but de ce TD est de comprendre comment le noyau gère la commutation de processus, les appels systèmes et les interruptions.

Prérequis

Vous devez connaître la différence entre mode utilisateur et mode système, être familiarisés avec la table des processus et connaître le rôle des variables `runrun`, `runin` et `runout`.

Commutation

Question 1

Quels sont les segments d'un processus ? Quels sont les segments du noyau ? Comment est organisée physiquement la mémoire associée ?

Question 2

Comment le noyau commute ?

Question 3

Expliquez l'algorithme de la fonction *switch* de commutation dans le noyau.

Entrée et sortie du système

Question 4

Que sont une trap, une exception et une interruption ? Décrivez la table des interruptions.

Question 5

Expliquez les actions effectuées lors d'un appel système.

Question 6

Expliquez les actions effectuées lors d'une interruption.

FICHER SWTCH.C

```

1  /*
2   * This routine is called to reschedule the CPU.
3   * if the calling process is not in RUN state ,
4   * arrangements for it to restart must have
5   * been made elsewhere , usually by calling via sleep .
6   */
7  swtch()
8  {
9      static struct proc *p;
10     register i, n;
11     register struct proc *rp;
12
13     if(p == NULL)
14         p = &proc[0];
15     /*
16      * Remember stack of caller
17      */
18     savu(u.u_rsav);
19     /*
20      * Switch to scheduler's stack
21      */
22     retu(proc[0].p_addr);
23
24     loop:
25         runrun = 0;
26         rp = p;
27         p = NULL;
28         n = 128;
29         /*
30          * Search for highest-priority runnable process
31          */
32         i = NPROC;
33         do {
34             rp++;
35             if(rp >= &proc[NPROC])
36                 rp = &proc[0];
37             if(rp->p_stat==SRUN && (rp->p_flag&SLOAD)!=0) {
38                 if(rp->p_pri < n) {
39                     p = rp;
40                     n = rp->p_pri;
41                 }
42             }
43         } while(--i);
44         /*
45          * If no process is runnable , idle .
46          */
47         if(p == NULL) {
48             p = rp;
49             idle();
50             goto loop;
51         }
52         rp = p;
53         /*
54          * Switch to stack of the new process and set up

```

```
55     * his segmentation registers.
56     */
57     retu(rp->p_addr);
58     sureg();
59     /*
60     * If the new process paused because it was
61     * swapped out, set the stack level to the last call
62     * to savu(u_ssav). This means that the return
63     * which is executed immediately after the call to aretu
64     * actually returns from the last routine which did
65     * the savu.
66     *
67     * You are not expected to understand this.
68     */
69     if(rp->p_flag&SSWAP) {
70         rp->p_flag =& ~SSWAP;
71         aretu(u.u_ssav);
72     }
73     /*
74     * The value returned here has many subtle implications.
75     * See the newproc comments.
76     */
77     return(1);
78 }
```

TD 6 - CRÉATION ET TERMINAISON DE PROCESSUS

Création de processus - fork

Question 1

Soit le programme C suivant :

```
int main(int argc, char *argv[]) {  
    int a = 10;  
    /* A COMPLETER */  
}
```

- Modifiez ce programme pour créer N processus fils. Chaque processus doit incrémenter la variable **a**, afficher la nouvelle valeur de **a** et se terminer. Le processus main attend la fin des N fils puis affiche **a** avant de se terminer.
- Quelles sont les valeurs affichées par chacun des processus ?

Question 2

L'appel système *fork* appelle la fonction *newproc* pour créer un process fils.

En analysant le code de *newproc* :

- Quelles sont les ressources (structures) qui seront partagées par le père et le fils ? Quels compteurs de référence seront alors modifiés ?
- Comment le manque de mémoire est-il géré ?

Question 3

Donnez l'algorithme de *fork* et *newproc*.

Terminaison de processus - exit

Question 4

Qu'est ce qu'un processus zombi ? Quand un processus n'est plus zombi ?

Question 5

Donnez l'algorithme du code d' *exit*. Quelle est la signification (l'utilité) du wakeup de la lignes 35 ?

Question 6

Les primitives *exit* et *wait* sont très liées. Donnez le code interne de la primitive *wait()* (sans prendre en compte les statistiques d'utilisation) en vous inspirant du code de *exit*.

FICHER FORK.C

```

1
2 fork ()
3 {
4     register proc *p1; *p2;
5
6
7     for (p2 = &proc[0]; p2 < &proc[NPROC]; p2++)
8         if (p2->p_stat == NULL)
9             goto found;
10    u.u_error = EAGAIN;
11    goto out;
12
13 found:
14     if (newproc()) {
15         u.u_ar0[R0] = 0;
16         u.u_cstime[0] = 0;
17         u.u_cstime[1] = 0;
18         u.u_stime = 0;
19         u.u_cutime[0] = 0;
20         u.u_cutime[1] = 0;
21         u.u_utime = 0;
22         return;
23     }
24
25     u.u_ar0[R0] = p2->p_pid;
26 out:
27 }
28
29
30 /* Create a new process (the internal version of fork )
31    The new process returns 1 in the new process.
32    The essential fact is that the new process is created in such
33    a way that it appears to have started executing in the
34    same call to newproc as the parent, but in fact the code runs is that of switch.
35    The subtle implication of the returned value of switch is that this is
36    the value that newproc's caller in the new process sees.*/
37
38 newproc ()
39 {
40     int a1, a2;
41     struct proc *p,*up;
42     register struct proc *rpp;
43     register *rip, n;
44     p = NULL;
45
46     /* First, just locate a slot for a process and copy the useful
47        info from this process into it. The panic 'cannot happen' because fork has
48        already checked for the existence of a slot. */
49
50    retry:
51        mpid++;
52        if (mpid < 0) {
53            mpid = 0;
54            goto retry;

```

```

55     }
56     for (rpp = &proc[0]; rpp < &proc[NPROC]; rpp++) {
57         if (rpp->p_stat == NULL && p == NULL)
58             p = rpp;
59         if (rpp->p_pid == mpid)
60             goto retry;
61     }
62
63     if ((rpp = p) == NULL)
64         panic("no_procs");
65
66     /* make proc entry for new proc */
67
68     rip = u.u_procp;
69     up = rip;
70     rpp->p_stat = SRUN;
71     rpp->p_flag = SLOAD;
72     rpp->p_uid = rip->p_uid;
73     rpp->p_ttyp = rip->p_ttyp;
74     rpp->p_nice = rip->p_nice;
75     rpp->p_textp = rip->p_textp;
76     rpp->p_pid = mpid;
77     rpp->p_ppid = rip->p_pid;
78     rpp->p_time = 0;
79
80     /* make duplicate entries where needed */
81
82     for (rip = &u.u_ofile[0]; rip < &u.u_ofile[NOFILE];)
83         if ((rpp = *rip++) != NULL)
84             rpp->f_count++;
85
86     if ((rpp = up->p_textp) != NULL) {
87         rpp->x_count++;
88         rpp->x_ccount++;
89     }
90
91     u.u_cdir->i_count++;
92
93     /* Partially simulate the environment of the new process so that
94        when it is actually created (by copying) it will look right */
95
96     savu(u.u_rsav);
97     rpp = p;
98     u.u_procp = rpp;
99     rip = up;
100    n = rip->p_size;
101    a1 = rip->p_addr;
102    rpp->p_size = n;
103    a2 = malloc(coremap, n);
104
105    /* if there is not enough memory for the new process,
106       swap out the current process to generate the copy */
107
108    if (a2 == NULL) {
109        rip->p_stat = SIDL;
110        rpp->p_addr = a1;
111        savu(u.u_ssav);
112        xswap(rpp, 0, 0);
113        rpp->p_flag = SSWAP;

```

```
114         rip-> p_stat = SRUN;
115     }
116     else {
117         /* there is memory, so just copy */
118
119         rpp-> p_addr = a2;
120         while (n--)
121             copyseg (a1++, a2++);
122     }
123
124     u.u_procp = rip;
125     return (0);
126 }
```

FICHER EXIT.C

```

1  exit()
2  {
3      register int *a, b;
4      register struct proc *p;
5
6      u.u_procp->p_flags &= ~STRC;
7      for (a = &u.u_signal[0]; a < &u.u_signal[NSIG];)
8          *a++ = 1;
9      for (a = &u.u_ofile[0]; a < &u.u_ofile[NOFILE]; a++)
10         if (b = *a) {
11             *a = NULL;
12             closef(b);
13         }
14
15
16     iput(u.u_cdir);
17     xfree();
18
19     b = malloc(swapmap, 1);
20     if (b == NULL)
21         panic("out_of_swap");
22
23     p = getblk(swapdev, b);
24     bcopy(&u, p->b_addr, 256);
25     bwrite(p);
26
27
28     a = u.u_procp;
29     mfree(coremap, a->p_size, a->p_addr);
30     a->p_addr = b;
31     a->p_stat = SZOMB;
32
33 loop:
34     for (p = &proc[0]; p < &proc[NPROC]; p++)
35         if (a->p_ppid == p->p_pid) {
36             wakeup(&proc[1]);
37             wakeup(p);
38             for (p = &proc[0]; p < &proc[NPROC]; p++)
39                 if (a->p_pid == p->p_ppid) {
40                     p->p_ppid = 1;
41                     if (p->p_stat == SSTOP)
42                         setrun(p);
43                 }
44             swtch();
45             /* no return */
46         }
47
48
49     a->p_ppid = 1;
50     goto loop;
51 }

```


TD 7 - LE SWAP

But

Le but de ce TD est de comprendre le fonctionnement du *swap* sous Unix à travers l'exemple d'une version simple.

Prérequis

Vous devez avoir assimilé les routines de synchronisation.

Question 1

Quelles sont les caractéristiques du processus 0 ($pid = 0$) ?

Question 2

Quelles sont les conditions qui provoquent la mise en attente du processus 0 ?

Expliquez le rôle des variables `runin` et `runout`.

Quels sont les critères qui provoquent l'éviction (*swap out*) d'un processus de la mémoire, le rappel (*swap in*) d'un processus en mémoire ?

Question 3

Décrivez l'algorithme de `sched`.

Question 4

Comment est réalisée l'entrée/sortie correspondant au swap ? Quelle est la structure de donnée utilisée ?

Où est situé l'espace de swap ?

Expliquez la procédure `swap`.

Question 5

La procédure **xswap** gère le cas de l'éviction en présence de segments *text* partagés. Quelles sont les actions effectuées par **xswap** ?

FICHER SCHED.C

```

1  /*
2  * The main loop of the scheduling (swapping) process.
3  * The basic idea is:
4  * see if anyone wants to be swapped in;
5  * swap out processes until there is room;
6  * swap him in;
7  * repeat.
8  * The runout flag is set whenever someone is swapped out.
9  * Sched sleeps on it awaiting work.
10 *
11 * Sched sleeps on runin whenever it cannot find enough
12 * memory (by swapping out or otherwise) to fit the
13 * selected swapped process. It is awakened when the
14 * memory situation changes and in any case once per second.
15 */
16
17
18 sched()
19 {
20     register struct proc *rp, *p;
21     register struct text *xp;
22     register int a, n;
23
24
25     /*
26      * find user to swap in;
27      * of users ready, select one out longest
28      */
29
30
31     loop:
32         spl(CLIHNB);
33         n = -1;
34         for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
35             if (rp->p_stat==SRUN && (rp->p_flag&SLOAD) == 0 &&
36                 rp->p_time > n) {
37                 p = rp;
38                 n = rp->p_time;
39             }
40         /*
41          * If there is no one there, wait.
42          */
43         if (n == -1) {
44             runout++;
45             sleep((caddr_t)&runout, PSWP);
46             goto loop;
47         }
48         spl(NORMAL);

```

```

49
50
51  /*
52   * See if there is memory for that process;
53   */
54
55
56  rp = p;
57  a = rp->p_size;
58  if ((xp=rp->p_textp) != NULL)
59      if (xp->x_ccount == 0)
60          a += xp->x_size;
61  if ((a=malloc(coremap, a)) != NULL)
62      goto found2;
63
64
65  /*
66   * none found.
67   * look around for easy memory.
68   * Select the largest of those sleeping
69   * at bad priority; if none, select the oldest.
70   */
71
72
73  spl(CLINHB);
74  for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
75      if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
76          (rp->p_stat==SSLEEP && rp->p_pri >= PZERO) || rp->p_stat==SSTOP))
77          goto found1;
78
79  if (n < 3)
80      goto sloop;
81  n = -1;
82  for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
83      if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
84          (rp->p_stat==SRUN || rp->p_stat==SSLEEP) &&
85          rp->p_time > n) {
86          p = rp;
87          n = rp->p_time;
88      }
89  if (n < 2)
90      goto sloop;
91  rp = p;
92
93
94  /*
95   * swap user out
96   */
97  found1:
98  spl(NORMAL);
99  rp->p_flag &= ~SLOAD;
100  xswap(rp, 1, 0);
101  goto loop;
102
103
104  /*
105   * swap user in
106   */
107  found2:

```

```

108     rp=p;
109     if ((xp=rp->p_textp) != NULL) {
110         if (xp->x_ccount == 0) {
111             if (swap(xp->x_daddr, a, xp->x_size, B_READ))
112                 goto swaper;
113             xp->x_caddr = a;
114             a += xp->x_size;
115         }
116         xp->x_ccount++;
117     }
118     if (swap(rp->p_addr, a, rp->p_size, B_READ))
119         goto swaper;
120     mfree(swapmap, (rp->p_size+7)/8, rp->p_addr);
121     rp->p_addr = a;
122     rp->p_flag |= SLOAD;
123     rp->p_time = 0;
124     goto loop;
125
126
127     sloop:
128         runin++;
129         sleep((caddr_t)&runin, PSWP);
130         goto loop;
131
132
133     swaper:
134         panic("swap_error");
135 }
136
137
138 /*
139  * swap IO header.
140  */
141 struct    buf        swbuf;
142
143
144 /*
145  * swap I/O
146  */
147 swap(blkno, coreaddr, count, rdflg)
148 caddr_t coreaddr;
149 {
150     register int c;
151     register lkflg;
152
153
154     spl(CLINHB);
155     while (swbuf.b_flags & B_BUSY) {
156         swbuf.b_flags |= B_WANTED;
157         sleep((caddr_t)&swbuf, PSWP);
158     }
159     swbuf.b_dev = swapdev;
160     swbuf.b_flags = B_BUSY | B_PHYS | rdflg;
161     swbuf.b_blkno = swplo+blkno;
162     swbuf.b_addr = coreaddr;
163     swbuf.b_count = count;
164     (*bdevsw[bmajor(swapdev)].d_strategy)(&swbuf);
165     spl(CLINHB);
166     while ((swbuf.b_flags & B_DONE) == 0)

```

```

167     sleep((caddr_t)&swbuf, PSWP);
168     if (swbuf.b_flags & B_WANTED)
169         wakeup((caddr_t)&swbuf);
170     spl(NORMAL);
171     swbuf.b_flags &= ~(B_BUSY|B_WANTED|B_PHYS);
172     return (swbuf.b_flags & B_ERROR);
173 }
174
175
176
177
178 /*
179  * Swap out process p.
180  * The ff flag causes its core to be freed—
181  * it may be off when called to create an image for a
182  * child process in newproc.
183  * Os is the old size of the data area of the process,
184  * and is supplied during core expansion swaps.
185  *
186  * panic: out of swap space
187  * panic: swap error — IO error
188 */
189
190
191 xswap(p, ff, os)
192 int *p;
193 {
194     register *rp, a;
195
196
197     rp = p;
198     if (os == 0)
199         os = rp->p_size;
200     a = malloc(swapmap, (rp->p_size+7)/8);
201     if (a == NULL)
202         panic("out_of_swap_space");
203     xccdec(rp->p_textp);
204     rp->p_flag |= SLOCK;
205     if (swap(a, rp->p_addr, os, 0))
206         panic("swap_error");
207     if (ff)
208         mfree(coremap, os, rp->p_addr);
209     rp->p_addr = a;
210     rp->p_flag &= ~(SLOAD|SLOCK);
211     rp->p_time = 0;
212     if (runout) {
213         runout = 0;
214         wakeup(&runout);
215     }
216 }

```

TD 8 - LE BUFFER CACHE

But

Le but de ce TD est l'étude du fonctionnement du *buffer cache* et l'optimisation de l'accès aux blocs.

Prérequis

Vous devez avoir compris le rôle de la fonction `bmap` et sa place dans le noyau.

Vous devez en outre connaître les différences entre *drivers* en mode *bloc* et en mode *caractère*.

Question 1

Quelles sont les différences entre les caches d'entrées / sorties sous Unix et les caches des processeurs ?

Question 2

Quel est le rôle du *buffer cache* ? Rappelez l'organisation du *buffer cache*. Comment les descripteurs sont-ils chaînés entre eux ?

Question 3

Qu'est-ce qu'un *device number* ? Quelle est la fonction qui traite l'entrée / sortie physique ?

Question 4

Que représentent les états `B_BUSY` et `B_DELWRI` ?

Expliquez pourquoi et quand un buffer ne se trouve plus dans la liste des buffers libres. Expliquez comment il y est remis. Commentez l'intérêt de l'opération.

Question 5

Expliquez ce qui se passe lorsqu'un processus demande une lecture et que :

1. le bloc est déjà dans un buffer,
2. le bloc n'est pas dans un buffer, et la *free-list* commence par un buffer non modifié, et
3. le bloc n'est pas dans un buffer, et la *free-list* commence par un buffer modifié.

Expliquez les avantages et inconvénients de l'utilisation du *buffer cache*.

Question 6

Quel est le rôle du flag `B_WANTED` ? Expliquez ce qui se passe lorsqu'un processus essaye de lire des données dans un fichier alors qu'une entrée / sortie est déjà en cours sur ce même bloc du même fichier ?

Commentez l'utilisation des deux routines `sleep` et `wakeup`.

Question 7

Que signifie l'état `B_DONE` ? Expliquez comment est effectué le contrôle de la fin de l'entrée / sortie. Décrivez le fonctionnement de la fonction `iodone()`.

Question 8

Décrivez l'algorithme de `getblk`.

Question 9

Complétez le corps de la fonction *brelse*, qui libère un tampon quand le noyau a fini de l'utiliser. La fonction doit réveiller les processus qui se sont endormis parce que le tampon était occupé et ceux qui se sont endormis parce qu'aucun tampon ne restait dans la liste de tampons libres. La fonction doit alors placer le tampon à la fin de la liste de tampons libres, à moins qu'une erreur d'entrée-sortie ne se soit produite. N'oubliez pas que la liste de blocs libres est une ressource critique et doit être accédée de façon exclusive (masquage/démasquage d'interruption).

Question 10

Complétez le corps de la fonction *bread* qui effectue la lecture d'un bloc disque. Cette fonction doit utiliser la fonction *getblk* pour rechercher le block dans le buffer cache. S'il y est, le système le lui retourne immédiatement sans le lire physiquement du disque. Sinon, *bread* doit appeler la fonction du périphérique disque qui lance la lecture d'un bloc. Dans ce cas, la fonction devra endormir le processus qui l'a appelée, qui sera réveillé par l'interruption disque.

Expliquez maintenant le code de la fonction *breada*, qui lit deux blocs, dont le deuxième de façon asynchrone. Quel est le but d'offrir une telle fonction ?

Question 11

Complétez le corps de la fonction *bwrite* qui effectue l'écriture d'un bloc disque. La fonction indique au périphérique du disque qu'il y a un tampon dont le contenu doit être enregistré sur le disque. Si l'écriture est synchrone, le processus appelant s'endort en attendant la fin de l'écriture. Puis il libère le bloc quand il est réveillé. Si l'écriture est asynchrone, la fonction lance l'écriture mais n'attend pas sa fin.

Expliquez le code de la fonction *bdwrite* et *bawrite*.

```

1  #include " ../sys/buf.h"
2  #include " ../sys/param.h"
3  #include " ../sys/types.h"
4
5
6  /*
7   * The following several routines allocate and free
8   * buffers with various side effects. In general the
9   * arguments to an allocate routine are a device and
10  * a block number, and the value is a pointer to
11  * to the buffer header; the buffer is marked "busy"
12  * so that no one else can touch it. If the block was
13  * already in core, no I/O need be done; if it is
14  * already busy, the process waits until it becomes free.
15  * The following routines allocate a buffer:
16  *     getblk
17  *     bread
18  *     breada
19  * Eventually the buffer must be released, possibly with the
20  * side effect of writing it out, by using one of
21  *     bwrite
22  *     bdwrite
23  *     bawrite
24  *     brelse
25  */
26
27
28  /*
29   * Unlink a buffer from the available list and mark it busy.
30   * (internal interface)
31   */
32  notavail(bp)
33  {
34      register s;
35
36
37      s = spl();
38      spl(BDINHB);
39      bp->av_back->av_forw = bp->av_forw;
40      bp->av_forw->av_back = bp->av_back;
41      bp->b_flags |= B_BUSY;
42      bfreelist.b_bcount--;
43      spl(s);
44  }
45
46
47  /*
48   * Read in (if necessary) the block and return a buffer pointer.
49   */
50  struct buf *
51  bread(dev, blkno)
52      dev_t dev;
53  daddr_t blkno;
54  {
55      register struct buf *bp;

```



```

56
57
58
59
60
61
62
63
64
65
66 }
67
68
69 /*
70  * Read in the block, like bread, but also start I/O on the
71  * read-ahead block (which is not allocated to the caller)
72  */
73 struct buf *
74 breada(dev, blkno, rablkno)
75     dev_t dev;
76     daddr_t blkno, rablkno;
77 {
78     register struct buf *bp, *rabp;
79
80
81     bp = NULL;
82     if (!incore(dev, blkno)) {
83         bp = getblk(dev, blkno);
84         if ((bp->b_flags & B_DONE) == 0) {
85             bp->b_flags |= B_READ;
86             bp->b_bcount = BSIZE;
87             (*bdevsw[bmajor(dev)].d_strategy)(bp);
88         }
89     }
90     if (rablkno && bfreelist.b_bcount > 1 && !incore(dev, rablkno)) {
91         rabp = getblk(dev, rablkno);
92         if (rabp->b_flags & B_DONE)
93             brelse(rabp);
94         else {
95             rabp->b_flags |= B_READ | B_ASYNC;
96             rabp->b_bcount = BSIZE;
97             (*bdevsw[bmajor(dev)].d_strategy)(rabp);
98         }
99     }
100     if (bp == NULL)
101         return(bread(dev, blkno));
102     iowait(bp);
103     return(bp);
104 }
105
106
107 /*
108  * Write the buffer, waiting for completion.
109  * Then release the buffer.
110  */
111 bwrite(bp)
112 register struct buf *bp;
113 {
114     register flag;

```

```
115
116
117
118
119
120
121
122
123
124
125
126 }
127
128
129 /*
130  * Release the buffer, marking it so that if it is grabbed
131  * for another purpose it will be written out before being
132  * given up (e.g. when writing a partial block where it is
133  * assumed that another write for the same block will soon follow).
134  * This can't be done for magtape, since writes must be done
135  * in the same order as requested.
136  */
137 bdwrite(bp)
138 register struct buf *bp;
139 {
140     bp->b_flags |= B_DELWRI | B_DONE;
141     bp->b_resid = 0;
142     brelse(bp);
143 }
144
145
146 /*
147  * Release the buffer, start I/O on it, but don't wait for completion.
148  */
149 bawrite(bp)
150 register struct buf *bp;
151 {
152     bp->b_flags |= B_ASYNC;
153     bwrite(bp);
154 }
155
156
157 /*
158  * release the buffer, with no I/O implied.
159  */
160 brelse(bp)
161 register struct buf *bp;
162 {
163
164
165
166
167
168
169
170
171
172
173
```

```
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191 }
192
193
194 /*
195  * See if the block is associated with some buffer
196  * (mainly to avoid getting hung up on a wait in breada)
197  */
198 incore(dev, blkno)
199 register dev_t dev;
200 daddr_t blkno;
201 {
202     register struct buf *bp;
203     register struct buf *dp;
204
205
206     dp = bhash(dev, blkno);
207     for (bp=dp->b_forw; bp != dp; bp = bp->b_forw)
208         if (bp->b_blkno==blkno && bp->b_dev==dev)
209             return(1);
210     return(0);
211 }
212
213
214 /*
215  * Assign a buffer for the given block. If the appropriate
216  * block is already associated, return it; otherwise search
217  * for the oldest non-busy buffer and reassign it.
218  */
219 struct buf *
220 getblk(dev, blkno)
221     register dev_t dev;
222     daddr_t blkno;
223 {
224     register struct buf *bp;
225     register struct buf *dp;
226
227
228     loop:
229         spl(NORMAL);
230         dp = bhash(dev, blkno);
231         if (dp == NULL)
232             panic("devtab");
```

```

233     for (bp=dp->b_forw; bp != dp; bp = bp->b_forw) {
234         if (bp->b_blkno!=blkno || bp->b_dev!=dev)
235             continue;
236         spl(BDINHB);
237         if (bp->b_flags&B_BUSY) {
238             bp->b_flags |= B_WANTED;
239             sleep((caddr_t)bp, PRIBIO+1);
240             goto loop;
241         }
242         spl(NORMAL);
243         notavail(bp);
244         return(bp);
245     }
246     spl(BDINHB);
247     if (bfreelist.av_forw == &bfreelist) {
248         bfreelist.b_flags |= B_WANTED;
249         sleep((caddr_t)&bfreelist, PRIBIO+1);
250         goto loop;
251     }
252     spl(NORMAL);
253     bp = bfreelist.av_forw;
254     notavail(bp);
255     if (bp->b_flags & B_DELWRI) {
256         bp->b_flags |= B_ASYNC;
257         bwrite(bp);
258         goto loop;
259     }
260     bp->b_flags = B_BUSY;
261     bp->b_back->b_forw = bp->b_forw;
262     bp->b_forw->b_back = bp->b_back;
263     bp->b_forw = dp->b_forw;
264     bp->b_back = dp;
265     dp->b_forw->b_back = bp;
266     dp->b_forw = bp;
267     bp->b_dev = dev;
268     bp->b_blkno = blkno;
269     return(bp);
270 }
271
272
273 /*
274  * Wait for I/O completion on the buffer; return errors
275  * to the user.
276  */
277 iowait(bp)
278 register struct buf *bp;
279 {
280
281
282     spl(BDINHB);
283     while ((bp->b_flags&B_DONE)==0)
284         sleep((caddr_t)bp, PRIBIO);
285     spl(NORMAL);
286     geterror(bp);
287 }
288
289
290 iodone(bp)
291 register buf *bp

```

```
292 {
293     bp->b_flags |= B_DONE;
294     if ((bp->b_flags & B_ASYNC) != 0)
295         brelse(bp);
296     else
297         wakeup((caddr_t)bp);
298 }
```

TD 9 - REPRESENTATION INTERNE DES FICHIERS

But

Le but de ce TD est d'étudier comment les fichiers et répertoires sont représentés et accédés dans le système UNIX.

Question 1

Ecrivez un programme C qui lit et affiche le contenu d'un fichier en utilisant les fonctions `open`, `read` et `close`.

Question 2

Comment la représentation interne d'un fichier est faite dans le système Unix? Et celle d'un répertoire?

Question 3

Pourquoi y-a-t-il une table des `inode` en mémoire?

Question 4

Etudiez les chaînages entre la table des *i_node* et la table *des fichiers*. Etudiez aussi le chaînage entre ces deux tables et la table *de descripteurs de fichier utilisateur*.

Expliquer le rôle des champs `i_count` et `f_count`.

Deux processus peuvent-ils ouvrir un même fichier? Un même fichier peut-il avoir deux ou plusieurs noms?

Question 5

Quelles sont les informations qu'une structure *i_node* contient?

Question 6

Décrivez l'algorithme de la fonction *iget*, qui rend la référence à un *i_node* dont le numéro est connu.

Question 7

Décrivez l'algorithme de la fonction *iput*, utilisée pour libérer un *i_node*.

Question 8

Comment le noyau affecte un *i_node* disque à un fichier nouvellement créé ? Comment le noyau gère les inodes libres dans le superbloc ?

Question 9

A quoi sert la fonction *namei* ? Donnez son pseudocode.

FICHER IGET.C

```

1  /* Look up an inode by device, inumber.
2    A pointer to a locked inode structure is returned
3    It does not include the mounting of volumes*/
4
5
6  iget (dev,ino)
7  {
8      register struct inode *p;
9      register *ip2;
10     int *ip1;
11
12
13     loop:
14         ip = NULL;
15         for (p = &inode[0]; p < &inode[NINODE]; p++){
16             if (dev == p->i_dev && ino == p->i_number ) {
17                 if ((p->i_flag & ILOCK) !=0) {
18                     p->i_flag |= IWANT;
19                     sleep (p,PINOD);
20                     goto loop;
21                 }
22
23
24                 p->i_count++;
25                 p->i_flag |= ILOCK;
26                 return (p);
27             }
28             if (ip== NULL && p->i_count ==0)
29                 ip =p;
30         }
31
32         if ((p = ip) == NULL) {
33             printf ("inode_table_overflow\n");
34             u.u_error = ENFILE;
35             return (NULL);
36         }
37
38
39         p->i_dev = dev;
40         p->i_number = ino;
41         p->i_flag = ILOCK;
42         p->i_count ++;
43         p->i_lastr =-1;
44         ip = bread (dev, ldiv (ino+31,16) );
45
46
47         /* check I/O errors */
48         if (ip->b_flags & B_ERROR) {
49             brelse (ip);
50             iput(p);
51             return (NULL);
52         }
53
54

```



```

55     ip1 = ip->b_addr + 32*lrem(ino+31,16) ;
56     ip2 = &p->i_mode ;
57
58     while (ip2 < &p->i_addr[8])
59         *ip2++ = *ip1++;
60     blrese (ip);
61     return (p);
62 }
63
64
65
66
67 /* Decrement reference count of an inode structure.
68 On the last reference, write the inode out and
69 if necessary, truncate and deallocate the file */
70
71 iput (p)
72 struct inode *p
73 {
74     register *rp;
75     rp =p;
76
77
78     if (rp ->i_count == 1) {
79         rp ->i_flag |= ILOCK;
80         if (rp ->i_nlink == 0) {
81             itrunc (rp);
82             rp->i_mode =0;
83             ifree (rp->i_dev , rp->i_number );
84
85
86         }
87         iupdate (rp, time);
88         prele(rp);
89
90     }
91     rp->i_count --;
92 }
93
94
95
96
97 ifree (dev,ino)
98 /* free the specified inode on the specified device */
99
100
101 iupdate (p,tm)
102 int *p, int *tm;{
103
104
105     /* check accessed and update flags on an inode
106 structure. If either is on, update the inode
107 with the corresponding dates set to the argument
108 tm */
109 }
110
111
112
113

```

```
114 itrunc (ip)
115 int ip;
116 {
117     /* free all disk blocks associated with the specified
118        inode structure. The blocks of the file are removed
119        in reverse order.
120    */
121 }
122
123
124 /*
125  * Unlock a pipe.
126  * If WANT bit is on,
127  * wakeup.
128  * This routine is also used
129  * to unlock inodes in general.
130 */
131 prele(ip)
132     register struct inode *ip;
133 {
134     ip->i_flag &= ~ILOCK;
135     if(ip->i_flag&IWANT) {
136         ip->i_flag &= ~IWANT;
137         wakeup((caddr_t)ip);
138     }
139 }
```

FICHER NAMEI.C

```

1  /*
2   * convert a pathname into a pointer to an inode.
3   * Note that the inode is locked
4   * func = function called to get next char of name
5   *      Uchar if name is in user space
6   * schar is name is in system space
7   *
8   * flag = 0, if name is sought
9   * 1 if name is to be created
10  * 2 if name is to be deleted
11  */
12
13
14
15
16 namei(func, flag)
17 int (*func) ();
18 {
19     register struct inode *dp;
20     register c;
21     register char *cp;
22     int eo, *bp;
23
24
25     /* if name starts with '/', start from root;
26        otherwise start from current dir; */
27
28
29     dp = u.u_cdir;
30     if ((c = (*func)()) == '/')
31         dp = rootdir;
32
33     iget(dp->i_dev, dp->i_number);
34     while (c == '/')
35         c = (*func)();
36     if (c == '\\0' && flag != 0) {
37         u.u_error = ENOENT;
38         goto out;
39     }
40
41
42     cloop:
43         /* Here dp contains pointer to last component matched */
44
45
46         if (u.u_error)
47             goto out;
48
49
50         if (c == '\\0')
51             return (dp);
52
53
54         /* if there is another component, dp must be a

```

```

55         directory and must have x permission */
56
57
58     if (( dp ->i_mode & IFMT) != IFDIR ) {
59         u.u_error = ENOTDIR;
60         goto out ;
61     }
62
63
64     if (access (dp, IEXEC ))
65         goto out;
66
67
68     /* gather up name into users' dir buffer */
69
70
71     cp = &u.u_dbuf[0];
72
73     while ( c!= '/' && c != '\0' && u.u_error == 0 ) {
74         if (cp < &u.u_dbuf [DIRSIZ] )
75             *cp++ =c;
76         c= (*func) () ;
77
78
79     }
80
81
82     while (cp < &u.u_dbuf[DIRSIZ])
83         *cp++ = '\0';
84
85
86     while (c == '/')
87         c= (*func) ();
88
89
90     if (u.u_error)
91         goto out;
92
93
94     /* set up to search a directory */
95     u.u_offset [1] = 0;
96     u.u_offset [0] = 0 ;
97     u.u_segflg=1;
98     eo = 0;
99     u.u_count = ldiv (dp->i_size , DIRSIZ +2 );
100    bp= NULL;
101
102
103    eloop:
104        /* if at the end of the directory, the search failed.
105        Report what is appropriate as per flag */
106        if (u.u_count == 0) {
107            if (bp != NULL)
108                brelse (bp);
109
110            if (flag == 1 && c == '/0' ) {
111                if (access (dp, IWRITE)
112                    goto out;
113

```

```

114         u_u_pdir = dp;
115         if (eo)
116             u.u_offset [1] = eo - DIRSIZE -2;
117         else
118             dp ->i_flag |= IUPD;
119         return (NULL);
120     }
121     u.u_error = ENOENT;
122     goto out;
123 }
124
125
126 /* if offset is on a block-boundary, read the next
127 directory block. Release previous if it exists */
128
129
130 if ((u.u_offset [1] & 0777 == 0){
131     if (bp != NULL )
132         brelse (bp);
133     bp = bread (dp-> i_dev, bmap (dp, ldiv (u.u_offset [1], 512 )));
134 }
135
136
137 /* Note first empty directory slot in eo
138 for possible creat. String compare the directory entry and
139 the current component. If they do not match, go back to sloop */
140
141
142 bcopy (bp->b_addr + (u.u_offset [1] & 0777), &u.u_dent,
143        (DIRSIZE +2)/2 );
144
145
146 u.u_offset [1] += DIRSIZ +2;
147
148
149 u.u_count --;
150
151
152 f (u.u_dent.u_ino == 0) {
153     if (eo == 0)
154         eo = u.u_offset [1];
155     goto eloop;
156 }
157
158
159 for (cp = &u.u_dbuf[0]; cp < &u.u_dbuf[DIRSIZ]; cp++)
160     if (*cp != cp[u.u_dent.u_name - u.u_dbuf] )
161         goto eloop;
162
163
164 /* here a component matched in a directory.
165 if there is more pathname, go back to eloop,
166 otherwise return */
167
168
169 if (bp != NULL)
170     brelse (bp);
171 if (flag == 2 && c == '\0') {
172     if (access (dp,IWRITE))

```

```
173             goto out;
174         return (dp);
175     }
176
177
178     bp = dp -> i_dev;
179     iput (dp);
180     dp = iget (bp, u.u_dent.u_ino);
181
182
183     if (dp == NULL)
184         return (NULL);
185     goto cloop;
186
187
188 out:
189     iput (dp);
190     return (NULL);
191 }
```

TD 10 - STRUCTURE DES FICHIERS

TRADUCTION D'ADRESSE / GESTION DE L'ESPACE LIBRE SUR DISQUE

But

Le but de ce TD est l'étude de la structure physique des fichiers sur disque : traduction d'adresse nécessaire entre l'adresse logique d'un bloc fourni par l'utilisateur et l'adresse physique sur disque et gestion de l'espace libre (allocation et libération de blocs).

Prérequis

Vous devez avoir utilisé et compris le fonctionnement des appels système **read**, **write** et **lseek**.

Vous devez avoir assimilé le fonctionnement des entrées/sorties sur Unix et le rôle des *inodes*.

Vous devez connaître la structure générale des disques et des volumes dans le système Unix, ainsi que la méthode d'accès aux données via le *buffer cache*.

Question 1

Rappelez l'implémentation physique des fichiers sous Unix.

Question 2

Quelle est la taille maximum d'un fichier ? Combien le noyau doit-il faire d'entrées / sorties au minimum et au maximum pour lire dans le fichier ?

Question 3

Quel est le rôle de la fonction **bmap** ? Expliquer son intérêt. Où se situe-t-elle dans le noyau par rapport à l'appel système ?

Question 4

Expliquez l'enchaînement des actions lorsqu'un utilisateur utilise les primitives **lseek** et **read**.

Question 5

Expliquez la gestion de l'espace libre.

Question 6

Quel est le rôle des fonctions `alloc` et `free` ?

A quels moments sont-elles appelées ? Donnez l'algorithme des fonctions `alloc` et `free`.

Question 7

Expliquez la provenance du champ `s_flock` du *super-block*.

FICHER SUBR.C

```

1  #include " ../sys/inode.h"
2  #include " ../sys/buf.h"
3  #include " ../sys/types.h"
4
5
6  /*
7   * Bmap defines the structure of file system storage
8   * by returning the physical block number on a device given the
9   * inode and the logical block number in a file.
10  * When convenient, it also leaves the physical
11  * block number of the next block of the file in rablock
12  * for use in read-ahead.
13  */
14
15
16  daddr_t
17  bmap(ip, bn, rwflg)
18      struct inode *ip;
19  daddr_t bn;
20  int rwflg;
21  {
22      register i;
23      struct buf *bp, *nbp;
24      int j, sh;
25      daddr_t nb, *bap;
26      dev_t dev;
27
28
29      if(bn < 0) {
30          u.u_error = EFBIG;
31          return((daddr_t)0);
32      }
33      dev = ip->i_dev;
34      rablock = 0;
35      /*
36       * blocks 0..NADDR-4 are direct blocks
37       */
38      if(bn < NADDR-3) {
39          i = bn;
40          nb = ip->i_addr[i];
41          if(nb == 0) {
42              if(rwflg==B_READ || (bp = alloc(dev))==NULL)
43                  return((daddr_t)-1);
44              nb = bp->b_blkno;
45              bdwrite(bp);
46              ip->i_addr[i] = nb;
47              ip->i_flag |= IUPD|ICHG;
48          }
49          if(i < NADDR-4)
50              rablock = ip->i_addr[i+1];
51          return(nb);
52      }
53      /*
54       * addresses NADDR-3, NADDR-2, and NADDR-1

```

```

55      * have single, double, triple indirect blocks.
56      * the first step is to determine
57      * how many levels of indirection.
58      */
59      sh = 0;
60      nb = 1;
61      bn -= NADDR-3;
62      for (j=3; j>0; j--) {
63          sh += NSHIFT;
64          nb <<= NSHIFT;
65          if (bn < nb)
66              break;
67          bn -= nb;
68      }
69      if (j == 0) {
70          u.u_error = EFBIG;
71          return((daddr_t)0);
72      }
73      /*
74       * fetch the address from the inode
75       */
76      nb = ip->i_addr[NADDR-j];
77      if (nb == 0) {
78          if (rwflg==B_READ || (bp = alloc(dev))==NULL)
79              return((daddr_t)-1);
80          nb = bp->b_blkno;
81          bdwrite(bp);
82          ip->i_addr[NADDR-j] = nb;
83          ip->i_flag |= IUPD|ICHG;
84      }
85      /*
86       * fetch through the indirect blocks
87       */
88      for (; j<=3; j++) {
89          bp = bread(dev, nb);
90          if (bp->b_flags & B_ERROR) {
91              brelse(bp);
92              return((daddr_t)0);
93          }
94          bap = bp->b_daddr;
95          sh -= NSHIFT;
96          i = (bn>>sh) & NMASK;
97          nb = bap[i];
98          if (nb == 0) {
99              if (rwflg==B_READ || (nbp = alloc(dev))==NULL) {
100                  brelse(bp);
101                  return((daddr_t)-1);
102              }
103              nb = nbp->b_blkno;
104              bdwrite(nbp);
105              bap[i] = nb;
106              bdwrite(bp);
107          } else
108              brelse(bp);
109      }
110      /*
111       * calculate read-ahead.
112       */
113      if (i < NINDIR-1)

```

```
114         rablock = bap[i+1];
115     return(nb);
116 }
```

FICHER ALLOC.C

```

1  #include " ../ sys / filsys . h "
2  #include " ../ sys / fblk . h "
3  #include " ../ sys / buf . h "
4  #include " ../ sys / inode . h "
5
6
7  typedef struct fblk *FBLKP;
8
9
10 /*
11  * alloc will obtain the next available
12  * free disk block from the free list of
13  * the specified device.
14  * The super block has up to NICFREE remembered
15  * free blocks; the last of these is read to
16  * obtain NICFREE more . . .
17  */
18 struct buf *
19 alloc (dev)
20     dev_t dev;
21 {
22     daddr_t bno;
23     register struct filsys *fp;
24     register struct buf *bp;
25
26     fp = getfs (dev);
27     while (fp->s_flock)
28         sleep ((caddr_t)&fp->s_flock, PINOD);
29     do {
30         if (fp->s_nfree <= 0)
31             goto nospace;
32         if (fp->s_nfree > NICFREE) {
33             prdev ("Bad_free_count", dev);
34             goto nospace;
35         }
36         bno = fp->s_free[--fp->s_nfree];
37         if (bno == 0)
38             goto nospace;
39     } while (badblock (fp, bno, dev));
40     if (fp->s_nfree <= 0) {
41         fp->s_flock++;
42         bp = bread (dev, bno);
43         if ((bp->b_flags & B_ERROR) == 0) {
44             fp->s_nfree = ((FBLKP) (bp->b_addr))->df_nfree;
45             bcopy ((caddr_t) ((FBLKP) (bp->b_addr))->df_free,
46                 (caddr_t) fp->s_free, sizeof (fp->s_free));
47         }
48         brelse (bp);
49         fp->s_flock = 0;
50         wakeup ((caddr_t)&fp->s_flock);
51         if (fp->s_nfree <= 0)
52             goto nospace;
53     }
54 }

```

```

55     if (fp->s_nfree <= 0 || fp->s_nfree > NICFREE) {
56         prdev("Bad_free_count", dev);
57         goto nospace;
58     }
59
60
61     bp = getblk(dev, bno);
62     clrbuf(bp);
63     if (fp->s_tfree) fp->s_tfree--;
64     fp->s_fmod = 1;
65     return(bp);
66
67
68 nospace:
69     fp->s_nfree = 0;
70     fp->s_tfree = 0;
71     prdev("no_space", dev);
72     u.u_error = ENOSPC;
73     return(NULL);
74 }
75
76
77 /*
78  * place the specified disk block
79  * back on the free list of the
80  * specified device.
81  */
82
83
84 free(dev, bno)
85 dev_t dev;
86 daddr_t bno;
87 {
88     register struct filsys *fp;
89     register struct buf *bp;
90
91
92     fp = getfs(dev);
93     while (fp->s_flock)
94         sleep((caddr_t)&fp->s_flock, PINOD);
95     if (badblock(fp, bno, dev))
96         return;
97     if (fp->s_nfree <= 0) {
98         fp->s_nfree = 1;
99         fp->s_free[0] = 0;
100    }
101    if (fp->s_nfree >= NICFREE) {
102        fp->s_flock++;
103        bp = getblk(dev, bno);
104        ((FBLKP)(bp->b_addr))->df_nfree = fp->s_nfree;
105        bcopy((caddr_t)fp->s_free,
106              (caddr_t)((FBLKP)(bp->b_addr))->df_free,
107              sizeof(fp->s_free));
108        fp->s_nfree = 0;
109        bwrite(bp);
110        fp->s_flock = 0;
111        wakeup((caddr_t)&fp->s_flock);
112    }
113    fp->s_free[fp->s_nfree++] = bno;

```

```
114     fp->s_tfree++;
115     fp->s_fmod = 1;
116 }
117
118
119 /*
120  * Check that a block number is in the
121  * range between the I list and the size
122  * of the device.
123  * This is used mainly to check that a
124  * garbage file system has not been mounted.
125  *
126  * bad block on dev x/y — not in range
127  */
128
129 badblock(fp, bn, dev)
130 register struct filsys *fp;
131 daddr_t bn;
132 dev_t dev;
133 {
134
135
136     if (bn < fp->s_iseize || bn >= fp->s_fsize) {
137         prdev("bad_block", dev);
138         return(1);
139     }
140     return(0);
141 }
142 }
```

TD 11 - COMPLÉMENT SUR LES ENTRÉES-SORTIES

But

Le but de ce TD est de comprendre les mécanismes mis en jeu dans un petit programme écrit en C, appelé *revision.c* utilisant un `fork()` (voir en annexe). On considère que les seuls processus s'exécutant dans le système sont le processus père et le processus fils décrits dans ce programme. Le nombre de buffers dans le buffer cache est suffisant pour contenir tous les blocs de fichier lus. Au démarrage, seul l'inode ROOT est chargée en RAM et aucun buffer n'est utilisé. On considère que les blocs font 512 octets, qu'ils peuvent contenir 16 inodes et qu'un inode contient 13 entrées de bloc : 10 entrées directes, une indirecte, une double indirecte et une triple indirecte. La figure 1 montre une partie du système de fichier et le chaînage entre les inodes, les blocs et les répertoires.

Prérequis

Vous devez avoir fait les TD sur les fichiers et le buffer cache.

Question 1

Donner une image de l'arborescence. Quels sont les liens durs ? Combien de blocs utilisent les fichiers ?

Question 2

Que produit le `printf()` de la ligne 29 ?

Question 3

Donnez la suite d'évènements entre le `open()` en mode U de la ligne 7 et le retour de la fonction, en passant par la gestion du buffer-cache.

Question 4

Combien d'entrée-sorties sont générées par le `open()` de la ligne 7 (on utilise `breada` pour les blocs de donnée et `bread` pour les blocs d'inodes) ? Donner une représentation des structures de données après l'appel (buffer-cache, tables des inodes, table des fichiers ouverts et structure `u_ofile`).

Question 5

Combien d'accès au disque sont générés par les `open()` aux lignes 8 et 18 ?

Question 6

Que fait le `lseek()` de la ligne 15 ? Combien alloue-t-il de bloc ? Que se passe-t-il si un processus lit les premiers octets du fichiers ?

Question 7

Combien d'entrées-sorties immédiates sont générées par les trois `write()` lignes 19, 20, 21 ? Que contiennent les nouveaux buffer du buffer cache ? (On peut partir de l'hypothèse que la table des blocs libres de la structure `filesys` contient les nombres suivants : 5678, 5679 et 5680).

Question 8

Combien d'entrées-sorties génèrent la lecture en ligne 24 ? Dessiner de nouveau les structures de données. Quelle est la sortie du `printf()` de la ligne 25 ?

Question 9

Quelle est la sortie de la ligne 28 ? Expliquer pourquoi.

Question 10

L'inode 130 se trouve-t-elle en RAM après le `close()` de la ligne 30 ? Combien d'entrée-sortie sont générées par cette ligne ?

Question 11

Peut-on être commuté pendant l'appel à `open()` de la ligne 18 ? Pourquoi ? Et pour le `write()` de la ligne 19 et le `open()` de la ligne 8 ?

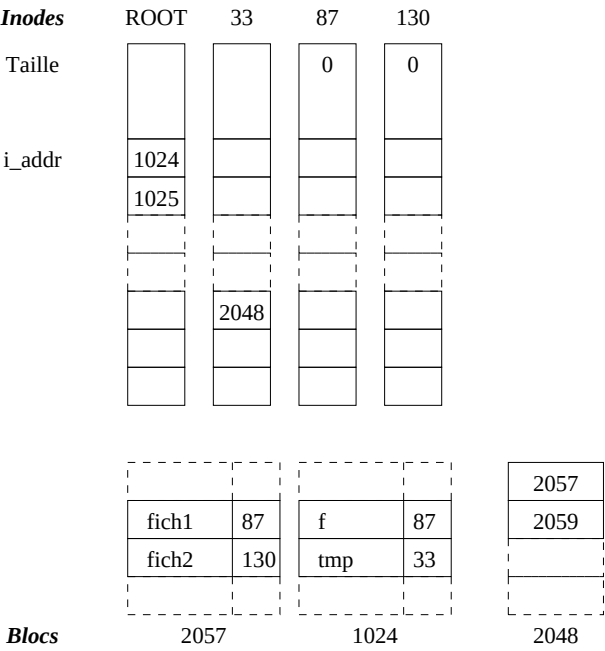


FIGURE 1 – État initial des inodes

FICHER REVISION.C

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdio.h>
5
6 int main() {
7     int a      = open("/tmp/fich1", O_RDWR);
8     int b      = open("/tmp/fich2", O_RDWR);
9     int c      = -1;
10    int status = 0;
11    char buf[3];
12
13    buf[2] = 0;
14
15    lseek(b, 16*512, SEEK_SET);
16
17    if(!fork()) {
18        c = open("/f", O_RDWR);
19        write(b, "hello", 5);
20        write(c, "abcd", 4);
21        write(a, "xy", 2);
22    } else {
23        wait(&status);
24        read(a, buf, 2);
25        printf("<%s>\n", buf);
26
27        read(b, buf, 2);
28        printf("<%s>\n", buf);
29        printf("c=%d\n", c);
30        close(b);
31    }
32
33    return 0;
34 }
```

ANNEXES DES TRAVAUX DIRIGÉS UNIX



DÉFINITION DES TYPES ET
STRUCTURES DE DONNÉES DU NOYAU

FICHER BUF.H

```

1  /*
2  * Each buffer in the pool is usually doubly linked into 2 lists :
3  * the device with which it is currently associated (always)
4  * and also on a list of blocks available for allocation
5  * for other use (usually).
6  * The latter list is kept in last-used order, and the two
7  * lists are doubly linked to make it easy to remove a buffer
8  * from one list when it was found by looking through the other.
9  * A buffer is on the available list, and is liable
10 * to be reassigned to another disk block, if and only
11 * if it is not marked BUSY. When a buffer is busy, the
12 * available-list pointers can be used for other purposes.
13 * Most drivers use the forward ptr as a link in their I/O active queue.
14 * A buffer header contains all the information required to perform I/O.
15 */
16 struct buf
17 {
18     int      b_flags;          /* buffer flags */
19     struct   buf *b_forw;      /* previous buf on b_list */
20     struct   buf *b_back;      /* next buf on b_list */
21     struct   buf *av_forw;     /* previous buf on av_list */
22     struct   buf *av_back;     /* next buf on av_list */
23     int      b_dev;           /* major+minor device name */
24     int      b_count;         /* transfer count */
25     union {
26         caddr_t b_un_addr;    /* low order core address */
27         struct   filsys *b_un_filsys; /* superblocks */
28         struct   dinode *b_un_dino; /* ilist */
29         daddr_t *b_un_daddr;  /* indirect block */
30     } b_un;
31     int      *b_xmem;         /* transfer memory address */
32     int      b_base;          /* page number for physical i/o */
33     int      b_size;          /* number of pages for physical i/o */
34     daddr_t  b_blkno;         /* block number on device */
35     char     b_error;         /* returned after I/O */
36     int      b_resid;         /* bytes not transfered */
37     int      b_pri;           /* Priority */
38 };
39 #define b_addr  b_un.b_un_addr
40 #define b_filsys b_un.b_un_filsys
41 #define b_dino  b_un.b_un_dino
42 #define b_daddr b_un.b_un_daddr
43
44
45 /*
46 * These flags are kept in b_flags.
47 */
48 #define B_WRITE 0x0000 /* non-read pseudo-flag */
49 #define B_READ  0x0001 /* read when I/O occurs */
50 #define B_DONE  0x0002 /* transaction finished */
51 #define B_ERROR 0x0004 /* transaction aborted */
52 #define B_BUSY  0x0008 /* not on av_forw/back list */
53 #define B_WANTED 0x0010 /* issue wakeup when BUSY goes off */
54 #define B_ASYNC 0x0020 /* don't wait for I/O completion */

```

```
55 #define B_PHYS 0x0040 /* wait I/O completion : physical I/O */
56 #define B_DELWRI 0x0080 /* don't write till block leaves avail list */
57
58
59 extern struct buf buf[]; /* The buffer pool itself */
60 extern struct buf bfreelist; /* head of available list */
61 extern char buffers[][BSIZE];
62
63
64 /*
65  * Fast access to buffers in cache by hashing.
66  */
67
68
69 #define bhash(d,b) ((struct buf *)&hbuf[((int)d+(int)b)&v.v_hmask])
70
71
72 struct hbuf
73 {
74     int b_flags;
75     struct buf *b_forw;
76     struct buf *b_back;
77 };
78
79
80 extern struct hbuf hbuf[];
```

FICHER CALLO.H

```
1  /*
2  *  The callout structure is for a routine arranging
3  *  to be called by the clock interrupt
4  *  (clock.c) with a specified argument,
5  *  in a specified amount of time.
6  */
7
8
9  struct   callo
10 {
11     int      c_time;           /* incremental time */
12     caddr_t  c_arg;           /* argument to routine */
13     int      (*c_func)();     /* routine */
14 };
15 struct   callo   callout [NCALL];
```

FICHER CONF.H

```
1  /*
2  *  Used to dissect integer device code
3  *  into major (driver designation) and
4  *  minor (driver parameter) parts.
5  */
6  struct
7  {
8      char    d_major;
9      char    d_minor;
10 };
11
12
13 /*
14 *  Declaration of device
15 *  switch. Each entry (row) is
16 *  the only link between the
17 *  main unix code and the driver.
18 *  The initialization of the
19 *  device switches is in the
20 *  file config.c.
21 *  Character device switch.
22 */
23 struct  cdevsw
24 {
25     int      (*d_open)();
26     int      (*d_close)();
27     int      (*d_read)();
28     int      (*d_write)();
29     int      (*d_xint)();
30     int      (*d_ioctl)();
31 } cdevsw [];
32
33
34 /*
35 *  Block device switch.
36 */
37 struct  bdevsw
38 {
39     int      (*d_open)();
40     int      (*d_close)();
41     int      (*d_strategy)();
42 } bdevsw [];
```

FICHIER FBLK.H

```
1 struct fblk
2 {
3     int df_nfree;
4     daddr_t df_free[NICFREE];
5 };
```


FICHER FILSYS.H

```

1  /*
2  *  Structure of the super-block
3  */
4  struct  filsys
5  {
6      unsigned short s_isize; /* size in blocks of i-list */
7      daddr_t s_fsize; /* size in blocks of entire volume */
8      short s_nfree; /* number of addresses in s_free */
9      daddr_t s_free[NICFREE]; /* free block list */
10     short s_ninode; /* number of i-nodes in s_inode */
11     ino_t s_inode[NICINOD]; /* free i-node list */
12     char s_flock; /* lock during free list manipulation */
13     char s_ilock; /* lock during i-list manipulation */
14     char s_fmod; /* super block modified flag */
15     char s_ronly; /* mounted read-only flag */
16     time_t s_time; /* last super block update */
17     daddr_t s_tfree; /* total free blocks */
18     ino_t s_tinode; /* total free inodes */
19     short s_m; /* interleave factor */
20     short s_n; /* " " */
21     char s_fname[6]; /* file system name */
22     char s_fpack[6]; /* file system pack name */
23
24
25     /* stuff for inode hashing */
26     ino_t s_lasti; /* start place for circular search */
27     ino_t s_nbehind; /* est # free inodes before s_last */
28 };
29
30
31 #define NICFREE 50
32 #define NICINOD 100

```

FICHER INODE.H

```

1  #define NADDR 13
2
3  struct inode
4  {
5      char          i_flag;
6      char          i_count; /* reference count */
7      dev_t         i_dev;   /* device where inode resides */
8      ino_t         i_number; /* i number, 1-to-1 with device address */
9      unsigned short i_mode;
10     short          i_nlink; /* directory entries */
11     short          i_uid;   /* owner */
12     short          i_gid;   /* group of owner */
13     off_t          i_size;  /* size of file */
14     struct {
15         daddr_t    i_addr[NADDR]; /* if normal file/directory */
16         daddr_t    i_lastr;        /* last logical block read (for read-ahead) */
17     };
18 };
19
20
21 extern struct inode inode[]; /* The inode table itself */
22
23 /* flags */
24 #define ILOCK  01 /* inode is locked */
25 #define IUPD   02 /* file has been modified */
26 #define IACC   04 /* inode access time to be updated */
27 #define IMOUNT 010 /* inode is mounted on */
28 #define IWANT  020 /* some process waiting on lock */
29 #define ITEXT  040 /* inode is pure text prototype */
30 #define ICHG0  100 /* inode has been changed */
31
32 /* modes */
33 #define IFMT    0170000 /* type of file */
34 #define IFDIR   0040000 /* directory */
35 #define IFCHR   0020000 /* character special */
36 #define IFBLK   0060000 /* block special */
37 #define IFREG   0100000 /* regular */
38 #define IFMPC   0030000 /* multiplexed char special */
39 #define IFMPB   0070000 /* multiplexed block special */
40 #define ISUID   04000   /* set user id on execution */
41 #define ISGID   02000   /* set group id on execution */
42 #define ISVTX   01000   /* save swapped text even after use */
43 #define IREAD   0400    /* read, write, execute permissions */
44 #define IWRITE  0200
45 #define IEXEC   0100

```

FICHER INODE.HV6

```

1  /*
2  * The I node is the focus of all
3  * file activity in unix. There is a unique
4  * inode allocated for each active file ,
5  * each current directory, each mounted-on
6  * file, text file, and the root. An inode is 'named'
7  * by its dev/inumber pair. (iget/iget.c)
8  * Data, from mode on, is read in
9  * from permanent inode on volume.
10 */
11 struct  inode
12 {
13     char    i_flag;
14     char    i_count;    /* reference count */
15     int i_dev;    /* device where inode resides */
16     int i_number;    /* i number, 1-to-1 with device address */
17     int i_mode;
18     char    i_nlink;    /* directory entries */
19     char    i_uid;    /* owner */
20     char    i_gid;    /* group of owner */
21     char    i_size0;    /* most significant of size */
22     char    *i_size1;    /* least sig */
23     int i_addr[8];    /* device addresses constituting file */
24     int i_lastr;    /* last logical block read (for read-ahead) */
25 } inode[NINODE];
26
27 /* flags */
28 #define ILOCK    01    /* inode is locked */
29 #define IUPD    02    /* inode has been modified */
30 #define IACC    04    /* inode access time to be updated */
31 #define IMOUNT    010    /* inode is mounted on */
32 #define IWANT    020    /* some process waiting on lock */
33 #define ITEXT    040    /* inode is pure text prototype */
34
35 /* modes */
36 #define IALLOC    0100000    /* file is used */
37 #define IFMT    060000    /* type of file */
38 #define IFDIR    040000    /* directory */
39 #define IFCHR    020000    /* character special */
40 #define IFBLK    060000    /* block special, 0 is regular */
41 #define ILARG    010000    /* large addressing algorithm */
42 #define ISUID    04000    /* set user id on execution */
43 #define ISGID    02000    /* set group id on execution */
44 #define ISVTX    01000    /* save swapped text even after use */
45 #define IREAD    0400    /* read, write, execute permissions */
46 #define IWRITE    0200
47 #define IEXEC    0100

```

FICHER INO.HV6

```
1
2 /*
3  * Inode structure as it appears on
4  * the disk. Not used by the system,
5  * but by things like check, df, dump.
6  */
7 struct inode
8 {
9     int i_mode;
10    char i_nlink;
11    char i_uid;
12    char i_gid;
13    char i_size0;
14    char *i_size1;
15    int i_addr[8];
16    int i_atime[2];
17    int i_mtime[2];
18 };
19
20 /* modes */
21 #define IALLOC 0100000
22 #define IFMT 060000
23 #define IFDIR 040000
24 #define IFCHR 020000
25 #define IFBLK 060000
26 #define ILARG 010000
27 #define ISUID 04000
28 #define ISGID 02000
29 #define ISVTX 01000
30 #define IREAD 0400
31 #define IWRITE 0200
32 #define IEXEC 0100
```

FICHER FILE.H

```
1  /*
2  * One file structure is allocated
3  * for each open/creat/pipe call.
4  * Main use is to hold the read/write
5  * pointer associated with each open
6  * file.
7  */
8  struct file
9  {
10     char    f_flag;
11     cnt_t    f_count;    /* reference count */
12     struct inode *f_inode;    /* pointer to inode structure */
13     long    f_offset;    /* read/write character index */
14 } file[NFILE];
15
16
17 /* flags */
18 #define FOPEN    (-1)
19 #define FREAD    0x0001
20 #define FWRITE    0x0002
21 #define FPIPE    0x0004
22
23
24 /* open parameters */
25 #define O_RDONLY    0
26 #define O_WRONLY    1
27 #define O_RDWR    2
```

FICHER MOUNT.H

```
1  /*
2   * Mount structure.
3   * One allocated on every mount.
4   */
5  struct  mount
6  {
7      int      m_flags;           /* status */
8      dev_t    m_dev;            /* device mounted */
9      struct  inode *m_inodp;    /* pointer to mounted on inode */
10     struct  buf *m_bufp;        /* buffer for super block */
11     struct  inode *m_mount;     /* pointer to mount root inode */
12 } mount[NMOUNT];
13
14
15 #define MFREE    0
16 #define MINUSE    1
17 #define MINTER    2
```

FICHER PARAM.H

```
1  /*
2   * fundamental constants
3   * cannot be changed
4   */
5
6
7  #define CBSIZE 12    /* number of info char in a clist block */
8  #define CROUND 15   /* sizeof(int *) + CBSIZE - 1 */
9  #define SROUND 7    /* CBSIZE>>1 */
10
11
12  /*
13   * processor priority levels
14   */
15  #define CLINHB 7     /* clock inhibit level */
16  #define BDINHB 6     /* block device inhibit level */
17  #define CDINHB 5     /* character device inhibit level */
18  #define CALOUT 4     /* clock callout processing level */
19  #define CDINTR 3     /* character device interrupt level */
20  #define WAKEUP 2     /* clock wakeup processing level */
21  #define SWITCH 1     /* switch processing level */
22  #define NORMAL 0     /* normal processing level */
23
24
25  #define NCALL 20     /* max simultaneous time callouts */
```

FICHIER PROC.H

```

1  /*
2  * One structure allocated per active process.
3  * It contains all data needed
4  * about the process while the
5  * process may be swapped out.
6  * Other per process data (user.h)
7  * is swapped with the process.
8  */
9
10
11 struct  proc
12 {
13     short   p_addr;    /* address of swappable image */
14     short   p_size;    /* size of swappable image (in blocks) */
15     int      p_flag;    /* process flags */
16     char     p_stat;    /* process state */
17     char     p_pri;    /* priority, negative is high */
18     char     p_nice;    /* nice for scheduling */
19     short    p_sig;    /* signals pending to this process */
20     /* signal number sent to this process */
21     short    p_uid;    /* real user id, used to direct tty signals */
22     short    p_suid;    /* set (effective) user id */
23     short    p_time;    /* resident time for scheduling */
24     int      p_cpu;    /* cpu usage for scheduling */
25     short    *p_ttyp;   /* controlling tty */
26     short    p_pid;    /* unique process id */
27     short    p_ppid;    /* process id of parent */
28     caddr_t  p_wchan;   /* event process is awaiting */
29     struct  text *p_textp; /* pointer to text structure */
30     short    p_tsize;   /* size of text */
31     short    p_ssize;   /* size of stack */
32     short    p_clktim;  /* time to alarm clock signal */
33 } proc [NPROC];
34
35
36 /* stat codes */
37 #define SSLEEP 1    /* awaiting an event */
38 #define SWAIT 2    /* (abandoned state) */
39 #define SRUN 3     /* running */
40 #define SIDL 4     /* intermediate state in process creation */
41 #define SZOMB 5     /* intermediate state in process termination */
42 #define SSTOP 6    /* process being traced */
43 #define SXBRK 7    /* process being xswapped */
44 #define SXSTK 8    /* process being xswapped */
45 #define SXTXT 9    /* process being xswapped */
46
47
48 /* flag codes */
49 #define SLOAD 0x0001 /* process in memory */
50 #define SSYS 0x0002  /* scheduling process */
51 #define SLOCK 0x0004 /* process locked in memory */
52 #define SSWAP 0x0008 /* process is being swapped out */
53 #define STRC 0x0010  /* process is being traced */
54 #define SWIED 0x0020 /* another tracing flag */

```



```
55 #define SMOVE    0x0040    /* process moved */
56 #define SULOCK    0x0080    /* user settable lock in core */
57 #define STEXT     0x0100    /* text pointer is valid */
58 #define SSPART     0x0200    /* process is partially swapped out */
```

FICHER SIGNAL.H

```
1 #define SIGHUP 1 /* hangup */
2 #define SIGINT 2 /* interrupt (rubout) */
3 #define SIGQUIT 3 /* quit (ASCII FS) */
4 #define SIGILL 4 /* illegal instruction (not reset when caught)*/
5 #define SIGTRAP 5 /* trace trap (not reset when caught) */
6 #define SIGIOT 6 /* IOT instruction */
7 #define SIGEMT 7 /* EMT instruction */
8 #define SIGFPE 8 /* floating point exception */
9 #define SIGKILL 9 /* kill (cannot be caught or ignored) */
10 #define SIGBUS 10 /* bus error */
11 #define SIGSEGV 11 /* segmentation violation */
12 #define SIGSYS 12 /* bad argument to system call */
13 #define SIGPIPE 13 /* write on a pipe with no one to read it */
14 #define SIGALRM 14 /* alarm clock */
15 #define SIGTERM 15 /* software termination signal from kill */
16 #define SIGUSR1 16 /* user defined signal 1 */
17 #define SIGUSR2 17 /* user defined signal 2 */
18 #define SIGCLD 18 /* death of a child */
19 #define SIGPWR 19 /* power-fail restart */
20
21
22 #define NSIG 20
23
24
25 #define SIG_DFL (int (*)(int))0
26 #define SIG_IGN (int (*)(int))1
```

FICHER TEXT.H

```
1  /*
2   * Text structure.
3   * One allocated per pure
4   * procedure on swap device.
5   * Manipulated by text.c
6   */
7  struct text
8  {
9      daddr_t x_daddr;           /* disk address of segment */
10     caddr_t x_caddr;          /* core address, if loaded */
11     long x_size;               /* size (*64) */
12     struct inode *x_iptr;      /* inode of prototype */
13     char x_count;              /* reference count */
14     char x_ccount;             /* number of loaded references */
15 } text[NTEXT];
```

FICHER TTY.H

```

1  /*
2  * A clist structure is the head
3  * of a linked list queue of characters.
4  * The characters are stored in 4-word
5  * blocks containing a link and 6 characters.
6  * The routines getc and putc (prim.c)
7  * manipulate these structures.
8  */
9  struct clist
10 {
11     int      c_cc;           /* character count */
12     char     *c_cf;          /* pointer to first character */
13     char     *c_cl;          /* pointer to last character */
14 };
15
16
17 struct cblock {
18     struct cblock *c_next;
19     char      c_info[CBSIZE];
20 };
21
22
23 struct    cblock    *cfreelis;
24
25
26 #define CBSIZE 12             /* nombre de caracteres par blocs */
27 #define CROUND 15             (sizeof(*int)+CBSIZE-1)
28 #define SROUND 7              (CBSIZE>>1)
29
30
31
32
33 /* Internal state bits */
34 #define CARR_ON 0x0001        /* Software copy of carrier present */
35 #define WOPEN 0x0002          /* Waiting for open to complete */
36 #define ISOPEN 0x0004         /* Device is open */
37 #define OPEN 0x0004
38 #define READING 0x0010        /* Input in progress */
39 #define WRITING 0x0020        /* Output in progress */
40 #define TTSTOP 0x0040         /* <^s><^q> processing */
41 #define TTSTART 0x0080        /* <^s><^q> processing */
42 #define TIMEOUT 0x0100        /* Delay timeout in progress */
43 #define ASLEEP 0x0200         /* Wakeup when output done */
44 #define XCLUDE 0x0400         /* exclusive use flag, against open */
45 #define HUPCLS 0x0800         /* Hangup after last close */
46 #define ATTENT 0x1000         /* Attention character received */
47 #define TBLOCK 0x2000         /* Tandem queue blocked */
48 #define CNTLQ 0x8000          /* interpret t_un as clist */

```

FICHER TYPES.H

```
1 typedef long          daddr_t  /* disk address */
2 typedef char *        caddr_t  /* core address */
3 typedef int           dev_t     /* device code */
4 typedef unsigned short ino_t    /* inode number */
```

FICHER USER.H

```

1  /*
2  * The user structure.
3  * One allocated per process.
4  * Contains all per process data
5  * that doesn't need to be referenced
6  * while the process is swapped.
7  * The user block is USIZE blocs
8  * long; resides at virtual kernel
9  * location 0xc000; contains the system
10 * stack per user; is cross referenced
11 * with the proc structure for the
12 * same process.
13 */
14 struct user
15 {
16     int      u_rsav[2];      /* saved env. for process switching */
17     int      u_ssav[2];      /* saved env. for swapping */
18     int      u_qsav[2];      /* saved env. for signaling */
19
20     struct   proc *u_procp;   /* pointer to proc structure */
21
22     char     u_error;         /* return error code */
23     char     u_intflg;        /* catch intr from sys */
24     int      *u_ar0;          /* address of users saved R0 */
25     int      u_arg[20];       /* arguments to current system call */
26     int      *u_ap;           /* pointer to arglist */
27
28     struct   file *u_ofile[NOFILE]; /* pointers to open file */
29
30     int      u_signal[NSIG];   /* disposition of signals */
31
32     int      u_uid;            /* effective user id */
33     int      u_gid;            /* effective group id */
34     int      u_ruid;           /* real user id */
35     int      u_rgid;           /* real group id */
36
37     int      u_uisa[16];       /* prototype of segmentation addresses */
38     int      u_uisd[16];       /* prototype of segmentation descriptors */
39
40     int      u_tsize;          /* text size (in blocs) */
41     int      u_dsize;          /* data size (in blocs) */
42     int      u_ssize;          /* stack size (in blocs) */
43     int      u_csize;          /* amount of stack in use (in blocs) */
44
45     long     u_utime;           /* this process user time */
46     long     u_stime;           /* this process system time */
47     long     u_cutime;          /* sum of childs' utimes */
48     long     u_cstime;          /* sum of childs' stimes */
49
50     int      u_segflg;         /* flag for i/o user or kernel */
51     char     *u_base;          /* base address for IO */
52     int      u_count;          /* bytes remaining for IO */
53     long     u_offset;         /* offset in file for IO */
54     struct   inode *u_cdir;     /* pointer to inode of current dir */

```

```

55     char    u_dbuf[DIRSIZ]; /* current pathname component */
56     char    *u_dirp;        /* current pointer to inode */
57     struct  {                /* current directory entry */
58         int    u_ino;
59         char    u_name[DIRSIZ];
60     } u_dent;
61     struct  inode *u_pdir;    /* inode of parent directory of dirp */
62                                     /* structures of open files */
63
64     int      u_stack[1]      /* kernel stack per user
65                               * extends from u + USIZE
66                               * backward not to reach here
67                               */
68 } u;
69
70
71 /* u_error codes */
72
73 #define EPERM    1           /* Not super-user */
74 #define ENOENT   2           /* No such file or directory */
75 #define ESRCH    3           /* No such process */
76 #define EINTR    4           /* interrupted system call */
77 #define EIO      5           /* I/O error */
78 #define ENXIO    6           /* No such device or address */
79 #define E2BIG     7           /* Arg list too long */
80 #define ENOEXEC   8          /* Exec format error */
81 #define EBADF     9           /* Bad file number */
82 #define ECHILD    10          /* No children */
83 #define EAGAIN    11          /* No more processes */
84 #define ENOMEM    12          /* Not enough core */
85 #define EACCES    13          /* Permission denied */
86 #define EFAULT    14          /* Bad address */
87 #define ENOTBLK   15          /* Block device required */
88 #define EBUSY     16          /* Mount device busy */
89 #define EEXIST     17          /* File exists */
90 #define EXDEV     18          /* Cross-device link */
91 #define ENODEV    19          /* No such device */
92 #define ENOTDIR   20          /* Not a directory */
93 #define EISDIR    21          /* Is a directory */
94 #define EINVAL    22          /* Invalid argument */
95 #define ENFILE    23          /* File table overflow */
96 #define EMFILE    24          /* Too many open files */
97 #define ENOTTY    25          /* Not a typewriter */
98 #define ETXTBSY   26          /* Text file busy */
99 #define EFBIG     27          /* File too large */
100 #define ENOSPC    28          /* No space left on device */
101 #define ESPIPE    29          /* Illegal seek */
102 #define EROFS     30          /* Read only file system */
103 #define EMLINK    31          /* Too many links */
104 #define EPIPE     32          /* Broken pipe */

```

FICHER VAR.H

```

1  /*
2   * The following is used by machdep.c
3   */
4  struct var {
5      int    v_uprocs;           /* max # of user's process */
6      int    v_timezone;        /* timezone */
7      int    v_cargs;           /* max # of bytes given to exec */
8      int    v_cspeed;          /* default asynchronous line speed */
9      long   v_fill[20];        /* rfu */
10     int     v_proc;            /* proc table */
11     struct  proc *ve_proc;
12     int     vs_proc;
13     int     v_clist;           /* cblock list */
14     struct  cblock *ve_clist;
15     int     vs_clist;
16     int     v_mount;          /* mount table */
17     struct  mount *ve_mount;
18     int     vs_mount;
19     int     v_inode;          /* inode table */
20     struct  inode *ve_inode;
21     int     vs_inode;
22     int     v_file;           /* file table */
23     struct  file *ve_file;
24     int     vs_file;
25     int     v_cmap;           /* core map */
26     struct  map *ve_cmap;
27     int     vs_cmap;
28     int     v_smap;           /* swap map */
29     struct  map *ve_smap;
30     int     vs_smap;
31     int     v_callout;        /* callout table */
32     struct  callo *ve_callout;
33     int     vs_callout;
34     int     v_text;           /* text segment table */
35     struct  text *ve_text;
36     int     vs_text;
37     int     v_buf;            /* data buffers */
38     struct  buf *ve_buf;
39     int     vs_buf;
40     /* beginning of internal buffers */
41     int     v_Buf;            /* data buffers */
42     struct  Buffer_Data *ve_Buf;
43     int     vs_Buf;
44     int     v_io;             /* space for io_info buf */
45     long    *ve_io;
46     int     vs_io;
47     int     v_hbuf;           /* structures for data buffers hashing */
48     struct  hbuf *ve_hbuf;
49     int     vs_hbuf;
50     int     v_hino;           /* structures for inode hashing */
51     struct  inode **ve_hino;
52     int     vs_hino;
53     int     v_hproc;          /* hash proc lists */
54     struct  proc **ve_hproc;

```



```
55     int      vs_hproc;
56     int      v_zero;
57     int      *ve_zero;
58     int      vs_zero;
59 } v;
60
61
62 struct proc *proc_end;  /* last logical proc of proc table */
63 long bufbase;
```